



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica

Lenguajes de Programación y Lab

Actividad Fundamental 6: Reporte de Lenguaje Funcional

Unidad 2

Semestre: 3ro

Equipo: 04

Grupo: 025

Salón: 3109

Periodo: Enero – Junio

Docente: Ing. Selene Guadalupe Pinal Torres

Hora: M4-M6

Nombre	Matricula	Carrera
Ruedas Vazquez Jordan Emanuel	2103677	ITS
Enríquez Nungaray Héctor Miguel	2042781	IAS
Hernández Silva Carlos Yahir	1975208	ITS
Heredia López Betsy Aracely	2054413	IAS

Viernes 11 de Abril del 2025

Monterrey, Nuevo León

Contenido

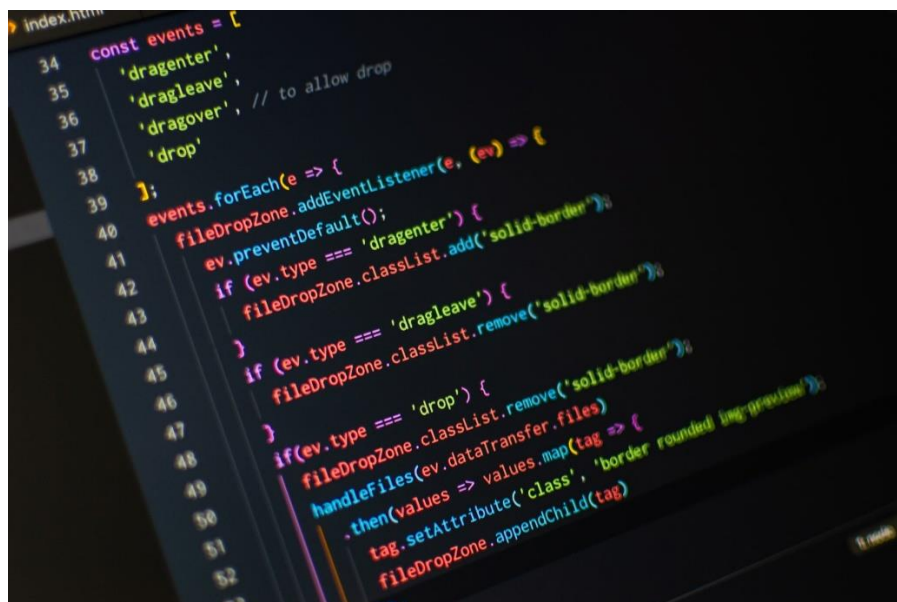
Introducción	2
Definición de lenguaje funcional	3
Lenguajes Funcionales	3
Aplicaciones Prácticas	4
Ejemplos	5
Conclusión General	11
Conclusiones Individuales.....	12
Referencias Bibliográficas	13

Introducción

La programación funcional es un paradigma de programación basado en el uso de funciones como elementos fundamentales para construir software. A diferencia de otros enfoques, como la programación imperativa u orientada a objetos, la programación funcional se centra en la evaluación de funciones puras, la inmutabilidad de los datos y la ausencia de efectos secundarios. Este enfoque permite escribir código más predecible, fácil de probar y mantener, lo que lo convierte en una opción atractiva para resolver problemas complejos, especialmente en entornos concurrentes y distribuidos.

La programación funcional se caracteriza por el uso de funciones puras, que garantizan resultados consistentes ante las mismas entradas sin generar efectos secundarios, evitando modificar variables externas o estados globales. Este paradigma promueve la inmutabilidad de los datos, donde las estructuras no se alteran, sino que se crean versiones nuevas con los cambios requeridos, mejorando la seguridad y predictibilidad del código. Las funciones son tratadas como ciudadanos de primera clase, pudiendo ser argumentos, retornos o almacenadas en estructuras, lo que favorece la flexibilidad y reutilización. Además, la transparencia referencial permite sustituir expresiones por sus valores sin afectar el programa, facilitando pruebas y optimizaciones, mientras que la evaluación perezosa (en lenguajes como Haskell) pospone cálculos hasta que son necesarios, optimizando recursos y permitiendo manejar datos infinitos.

A diferencia de paradigmas como el POO o imperativo, que se basan en estados mutables y objetos que combinan datos y comportamiento, la programación funcional evita efectos secundarios y centra su lógica en transformaciones predecibles de datos. Mientras en enfoques tradicionales se usan bucles y condicionales para controlar el flujo, aquí se privilegia la recursión y funciones de orden superior como *map*, *filter* o *reduce*. Además, frente a la modificación directa de variables comunes en POO, este paradigma prioriza la creación de nuevas instancias, reduciendo riesgos de inconsistencias y simplificando el razonamiento sobre el comportamiento del sistema.



Definición de lenguaje funcional

Es un paradigma de programación basado en el uso de funciones como el modelo central para construir software. Se inspira en las funciones matemáticas, donde cada conjunto de entradas produce una única salida, sin depender de estados externos. A diferencia de la programación imperativa, se enfoca en qué se quiere lograr, no en cómo hacerlo. Evita el uso de datos mutables, efectos secundarios y estados compartidos, permitiendo así mayor claridad, predictibilidad y facilidad para probar el código.

Principios clave:

- Cálculo lambda: Todo se representa usando funciones matemáticas.
- Declarativo: Se dice qué hacer, no cómo hacerlo paso a paso.
- Funciones puras: Siempre dan el mismo resultado sin cambiar nada fuera.
- Funciones de primera clase: Se pueden usar como valores, pasarlas o devolverlas.
- Inmutabilidad: Las variables no cambian una vez creadas.
- Sin efectos secundarios: No afecta nada fuera de la función.
- Recursión: Usa funciones que se llaman a sí mismas en lugar de bucles.
- Composición: Las funciones se pueden unir para formar otras más complejas.

Inmutabilidad:

Las variables no pueden ser modificadas una vez que se han inicializado. Si se requiere almacenar un nuevo valor, se define una nueva variable. Esta propiedad garantiza que el estado del programa no cambiará inesperadamente, lo cual mejora la confiabilidad y facilita el razonamiento del código.

Funciones puras:

Son funciones que, al recibir los mismos parámetros, siempre devuelven el mismo resultado y no alteran el estado del sistema ni tienen efectos secundarios, como modificar variables externas o interactuar con el sistema de archivos. Esto las hace más fáciles de probar, componer y mantener.

Recursión:

En lugar de usar bucles como for o while, la programación funcional utiliza funciones recursivas que se llaman a sí mismas hasta alcanzar un caso base. La recursión es el mecanismo natural para iterar en este paradigma.

Funciones de orden superior:

Son funciones que pueden recibir otras funciones como parámetros o devolver funciones como resultado. Esto permite construir operaciones complejas de forma más expresiva y concisa. Por ejemplo, funciones como filter, map o reduce en muchos lenguajes funcionales permiten trabajar con colecciones de datos sin necesidad de ciclos explícitos.



Lenguajes Funcionales

Los lenguajes funcionales ofrecen enfoques únicos para resolver problemas, cada uno con fortalezas y desafíos específicos. Haskell destaca por su código limpio y conciso, gracias a una sintaxis minimalista y un fuerte énfasis en la programación pura (sin efectos secundarios), lo que facilita la escritura de programas predecibles y mantenibles. Su evaluación perezosa permite optimizar recursos al retrasar cálculos hasta que son necesarios, ideal para manejar datos infinitos o operaciones costosas. Sin embargo, su curva de aprendizaje es empinada debido a conceptos avanzados como mónadas o tipos de datos algebraicos, y su comunidad, aunque activa, es más pequeña que la de lenguajes como Python o Java, lo que puede limitar la disponibilidad de bibliotecas o soporte inmediato.

Erlang, diseñado para sistemas concurrentes y distribuidos, brilla en entornos donde la tolerancia a fallos y la escalabilidad son críticas, como telecomunicaciones o aplicaciones en tiempo real. Su modelo de actores permite manejar miles de procesos ligeros simultáneamente sin bloquear el sistema. No obstante, su sintaxis, basada en Prolog, puede resultar poco intuitiva para programadores acostumbrados a lenguajes como C o Python, y su rendimiento en tareas computacionalmente intensivas (como procesamiento numérico) es inferior al de lenguajes compilados como C++ o Rust.

Clojure, al operar sobre la JVM, combina la inmutabilidad y concurrencia de la programación funcional con acceso al ecosistema de Java, ideal para aplicaciones empresariales. Su sintaxis basada en Lisp es simple y uniforme, pero el uso excesivo de paréntesis puede intimidar a nuevos desarrolladores. Aunque es menos popular que Java, su enfoque en la programación funcional pragmática lo hace poderoso para proyectos que requieren alta escalabilidad. Scala, también en la JVM, fusiona programación funcional y orientada a objetos, ofreciendo flexibilidad, pero su versatilidad puede generar complejidad en proyectos grandes, y su tiempo de compilación suele ser más lento. F#, integrado con .NET, destaca por una sintaxis clara y herramientas robustas en Visual Studio, aunque su adopción fuera del entorno Microsoft es limitada, y la documentación es menos extensa comparada con lenguajes como C#.

[illegible]

Aplicaciones Prácticas

Los lenguajes funcionales, como Haskell, Erlang o Scala, destacan en escenarios donde la **inmutabilidad de datos**, la **transparencia referencial** y el **manejo de operaciones complejas** son críticos. Su enfoque en funciones puras (sin efectos secundarios) y recursividad los hace ideales para áreas como el **procesamiento de datos** y la **inteligencia artificial (IA)**. En el procesamiento de datos, por ejemplo, lenguajes como Clojure o F# permiten manipular grandes volúmenes de información de forma eficiente, gracias a su capacidad para paralelizar tareas y evitar inconsistencias en flujos de trabajo. Un caso emblemático es el uso de **MapReduce** en Hadoop, inspirado en principios funcionales para distribuir cálculos en clusters.

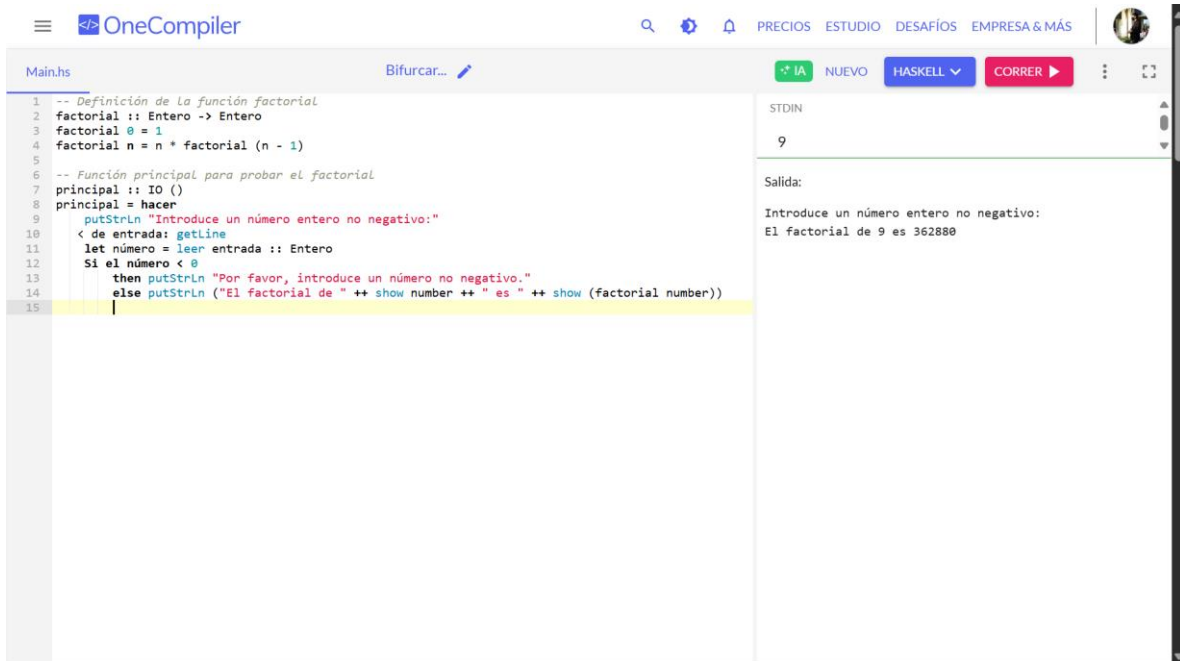
En **inteligencia artificial**, los lenguajes funcionales brindan ventajas únicas. Lisp, uno de los pioneros en este campo, se utilizó históricamente para desarrollar sistemas expertos y algoritmos de procesamiento simbólico debido a su flexibilidad para manipular estructuras de datos complejas. Hoy, Scala y Haskell son populares en **aprendizaje automático** (*machine learning*), donde la capacidad de escribir código conciso y libre de efectos secundarios simplifica la implementación de modelos estadísticos. Herramientas como **Apache Spark**, escrita en Scala, aprovechan la programación funcional para procesar datos distribuidos en tiempo real, esencial en análisis predictivos y entrenamiento de redes neuronales.

Más allá de estas áreas, los lenguajes funcionales son clave en **sistemas concurrentes y distribuidos**. Erlang, diseñado para telecomunicaciones, gestiona millones de conexiones simultáneas en aplicaciones como WhatsApp o RabbitMQ, gracias a su modelo de actores y tolerancia a fallos. En **finanzas**, donde la precisión es vital, lenguajes como OCaml se emplean para construir motores de trading algorítmico que evitan errores por estados compartidos. Además, en **desarrollo de compiladores** o **verificación formal**, lenguajes como Agda o Idris permiten demostrar matemáticamente la corrección del código, algo crucial en industrias como la aeroespacial. Aunque menos comunes en aplicaciones de bajo nivel, los lenguajes funcionales están ganando terreno en **frontend moderno**. Bibliotecas como React (JavaScript) adoptan principios funcionales para manejar estados predecibles, mientras que Elm ofrece un enfoque puro para construir interfaces web robustas. Esta diversidad de aplicaciones demuestra que la programación funcional no es un nicho académico, sino una herramienta versátil para resolver problemas reales con elegancia y eficiencia.



Ejemplos:

Ruedas Vazquez Jordan Emanuel



The screenshot shows the OneCompiler web IDE interface. The editor displays a Haskell program for calculating factorials. The code is as follows:

```
1 -- Definición de la función factorial
2 factorial :: Entero -> Entero
3 factorial 0 = 1
4 factorial n = n * factorial (n - 1)
5
6 -- Función principal para probar el factorial
7 principal :: IO ()
8 principal = hacer
9   putStrLn "Introduce un número entero no negativo:"
10  < de entrada: getLine
11  let número = leer entrada :: Entero
12  Si el número < 0
13    then putStrLn "Por favor, introduce un número no negativo."
14    else putStrLn ("El factorial de " ++ show number ++ " es " ++ show (factorial number))
15
```

On the right side, the 'STDIN' input shows the number '9'. The 'Salida:' (Output) section shows the program's execution: 'Introduce un número entero no negativo:' followed by 'El factorial de 9 es 362880'.

Código:

-- Definición de la función factorial

```
factorial :: Integer -> Integer
```

```
factorial 0 = 1
```

```
factorial n = n * factorial (n - 1)
```

-- Función principal para probar el factorial

```
main :: IO ()
```

```
main = do
```

```
    putStrLn "Introduce un número entero no negativo:"
```

```
    input <- getLine
```

```
    let number = read input :: Integer
```

```
    if number < 0
```

```
        then putStrLn "Por favor, introduce un número no negativo."
```

```
        else putStrLn ("El factorial de " ++ show number ++ " es " ++ show (factorial number))
```

Descripción del Código: El código en Haskell define una función recursiva factorial, donde $\text{factorial } 0 = 1$ es el caso base (el factorial de 0 es 1) y $\text{factorial } n = n * \text{factorial } (n - 1)$ calcula el factorial de un número multiplicándolo por el factorial del número anterior hasta llegar a 0. La función `main` solicita al usuario un número no negativo, convierte la entrada a entero y, si es válido (≥ 0), muestra el resultado aplicando la función factorial. Este enfoque refleja la elegancia de la programación funcional para resolver problemas con recursión y validación clara, manteniendo un flujo seguro y predecible desde la entrada hasta la salida.

Enríquez Nungaray Héctor Miguel

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>Calculadora de Áreas</title>
5 <style>
6   body {
7     font-family: Arial;
8     background-color: #f2f2f2;
9     padding: 20px;
10  }
11  input, button {
12    padding: 10px;
13    margin: 5px 0;
14    font-size: 16px;
15  }
16  .resultado {
17    margin-top: 20px;
18    background-color: #fff;
19    padding: 15px;
20    border-radius: 8px;
21    box-shadow: 0 2px 6px rgba(0,0,0,0.1);
22  }
23 </style>
24 </head>
25 <body>
26
27 <h2>📐 Calculadora de Áreas</h2>
28
29 <label>Base:</label><br>
30 <input type="number" id="base"><br>
31
32 <label>Altura:</label><br>
33 <input type="number" id="altura"><br>
34
35 <label>Radio:</label><br>
36 <input type="number" id="radio"><br><br>
37
38 <button onclick="calcularAreas()">Calcular Áreas</button>
39
40 <div class="resultado" id="resultado"></div>
41
42 <script>
43   function calcularAreaRectangulo(base, altura) {
44     return base * altura;
45   }
46
47   function calcularAreaTriangulo(base, altura) {
48     return (base * altura) / 2;
49   }
50
51   function calcularAreaCirculo(radio) {
52     return Math.PI * Math.pow(radio, 2);
53   }
54
55   function calcularAreas() {
56     let base = parseFloat(document.getElementById("base").value);
57
58     let altura = parseFloat(document.getElementById("altura").value);
59     let radio = parseFloat(document.getElementById("radio").value);
60
61     if (isNaN(base) || isNaN(altura) || isNaN(radio)) {
62       document.getElementById("resultado").innerHTML = " ! Por favor, ingresa todos los valores.";
63       return;
64     }
65
66     let areaRect = calcularAreaRectangulo(base, altura);
67     let areaTri = calcularAreaTriangulo(base, altura);
68     let areaCirc = calcularAreaCirculo(radio);
69
70     document.getElementById("resultado").innerHTML = `
71     ♦ Área del Rectángulo: <b>${areaRect.toFixed(2)}</b><br>
72     ♦ Área del Triángulo: <b>${areaTri.toFixed(2)}</b><br>
73     ♦ Área del Círculo: <b>${areaCirc.toFixed(2)}</b>
74   `;
75 </script>
76
77 </body>
78 </html>
```

 **Calculadora de Áreas**

Base:

Altura:

Radio:

- ♦ Área del Rectángulo: **300.00**
- ♦ Área del Triángulo: **150.00**
- ♦ Área del Círculo: **11309.73**

Explicación del Código Este código es una calculadora de áreas hecha con HTML y JavaScript. El usuario escribe la base, altura y radio en tres cajitas, y al hacer clic en el botón:

1. Se leen los valores ingresados.
2. Se calculan:
3. El área de un rectángulo ($\text{base} \times \text{altura}$)
4. El área de un triángulo ($(\text{base} \times \text{altura}) \div 2$)
5. El área de un círculo ($\pi \times \text{radio}^2$)
6. Se muestran los resultados en pantalla con dos decimales

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Ejemplo Funcional con HTML y JavaScript</title>
6 </head>
7 <body>
8   <h2>Números originales:</h2>
9   <ul id="original-list"></ul>
10
11  <h2>Números duplicados (función pura):</h2>
12  <ul id="doubled-list"></ul>
13
14  <script>
15    // Datos originales (inmutables)
16    const numeros = [1, 2, 3, 4, 5];
17
18    // Función pura que duplica un número
19    const duplicar = x => x * 2;
20
21    // Usamos map (función de orden superior) para apli
22    const duplicados = numeros.map(duplicar);
23
24    // Mostrar resultados en HTML
25    const mostrarLista = (elementoId, datos) => {
26      const ul = document.getElementById(elementoId);
27      datos.forEach(num => {
28        const li = document.createElement('li');
29        li.textContent = num;
30        ul.appendChild(li);
31      });
32    };
33
34    // Mostrar ambas listas
35    mostrarLista("original-list", numeros);
36    mostrarLista("doubled-list", duplicados);
37  </script>
38 </body>
39 </html>
40
41
```

Números originales:

- 1
- 2
- 3
- 4
- 5

Números duplicados (función pura):

- 2
- 4
- 6
- 8
- 10

Explicación del Código:

Se crean dos listas vacías en el HTML con identificadores (id) para poder llenarlas desde JavaScript. Una lista mostrará los números originales, y la otra los duplicados. Se define un arreglo con números del 1 al 5. La variable `numeros` es constante (inmutable), lo cual es una buena práctica en programación funcional. Esta es una función pura: siempre que le pases el mismo valor `x`, te devolverá el mismo resultado ($x * 2$). No modifica ninguna variable externa ni produce efectos secundarios. Aquí usamos `map`, una función de orden superior que recorre el arreglo `numeros` y aplica la función `duplicar` a cada elemento. El resultado es un nuevo arreglo `duplicados` sin modificar el original. Esta función recibe un id de lista (`ul`) y un arreglo de datos. Usa `forEach` para recorrer los datos y crear elementos

para mostrarlos en la página. Aunque interactúa con el DOM (efecto secundario), la transformación de los datos ocurre de forma pura antes

```
1 from functools import reduce
2
3 # Definir algunas funciones puras
4 square = lambda x: x * x
5 is_even = lambda x: x % 2 == 0
6 add = lambda x, y: x + y
7
8 # Lista de números para trabajar
9 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
10
11 # Aplicar transformaciones funcionales encadenadas:
12 # 1. Filtrar solo números pares
13 # 2. Elevar al cuadrado cada número
14 # 3. Sumar todos los resultados
15 result = reduce(add, map(square, filter(is_even, numbers)))
16
17 print(f"Números originales: {numbers}")
18 print(f"Resultado final (suma de cuadrados de números pares): {result}")
19
```

Nombre	Tipo	Tamaño	Valor
numbers	list	10	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
result	int	1	220

```
***
SyntaxError: invalid syntax

In [5]: %runfile "C:/Users/carlo/OneDrive/Documentos/Sin título1.py" --wdir
Números originales: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Resultado final (suma de cuadrados de números pares): 220

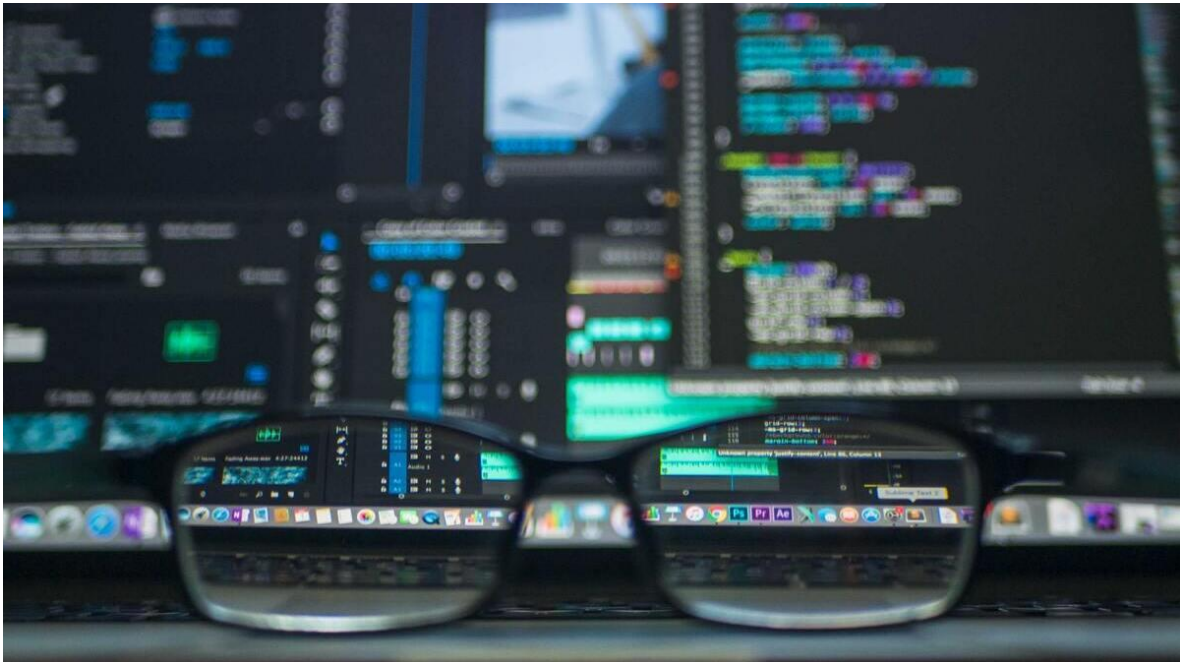
In [6]:
```

- Este código usa programación funcional para trabajar con una lista de números.
- Filtra los pares,
- eleva al cuadrado cada uno,
- y suma los resultados con reduce.
- Todo con funciones puras (lambda). El resultado es la suma de los cuadrados de los números pares.

Conclusión General

La programación funcional representa un enfoque poderoso y cada vez más relevante en el desarrollo de software, especialmente en contextos que requieren claridad, mantenibilidad y concurrencia. Al centrarse en funciones puras, la inmutabilidad y la ausencia de efectos secundarios, este paradigma permite escribir código más predecible, modular y fácil de probar. Aunque difiere notablemente de paradigmas como la programación orientada a objetos, ambos enfoques no son excluyentes y pueden complementarse según las necesidades del proyecto. Comprender las características, ventajas y limitaciones de la programación funcional permite a los desarrolladores tomar decisiones más informadas y mejorar la calidad de sus aplicaciones.

Los lenguajes de programación han evolucionado a lo largo de la historia para facilitar la interacción entre los programadores y las computadoras, permitiendo el desarrollo de software más eficiente y accesible. Desde los primeros lenguajes de bajo nivel como el ensamblador hasta las modernas tecnologías de alto nivel como Python y JavaScript, el objetivo ha sido siempre mejorar la productividad y la capacidad de resolución de problemas. Gracias a los avances en los paradigmas de programación, hoy en día existen lenguajes especializados para diversas áreas, desde la inteligencia artificial hasta el desarrollo web, cada uno optimizado para tareas específicas. Uno de los aspectos más importantes en la programación es la manera en que los lenguajes traducen las instrucciones humanas a código máquina. En este sentido, los compiladores e intérpretes juegan un papel crucial en la ejecución del software. Mientras que los compiladores generan programas altamente optimizados y rápidos, los intérpretes brindan mayor flexibilidad y facilidad de depuración.



Conclusiones Individuales

Ruedas Vazquez Jordan Emanuel

Los lenguajes funcionales, históricamente relegados a ámbitos académicos, han demostrado ser pilares en la resolución de problemas modernos gracias a su enfoque en la pureza, la inmutabilidad y la concurrencia segura. Su capacidad para manejar big data con eficiencia, construir sistemas distribuidos resilientes y garantizar precisión en campos críticos como las finanzas o la IA revela que no son solo herramientas técnicas, sino paradigmas que priorizan la claridad y la prevención de errores. Más allá de sus ventajas prácticas, nos enseñan una lección fundamental: en un mundo tecnológico complejo, la elegancia de una solución radica en su simplicidad y en cómo aborda los desafíos desde principios sólidos, no solo en su velocidad o popularidad. Su adopción creciente en industrias diversas no es una moda, sino un recordatorio de que, ante problemas cada vez más interdependientes, la programación funcional ofrece un marco para pensar de manera más estructurada, colaborativa y, en última instancia, más humana.

Enríquez Nungaray Héctor Miguel

Los lenguajes de scripting son eficientes para automatizar tareas y desarrollar aplicaciones rápidamente gracias a su simplicidad, mientras que los lenguajes funcionales se destacan por su enfoque en funciones puras e inmutabilidad, lo que favorece la seguridad, la legibilidad del código y el manejo del paralelismo. Ambos tipos de lenguajes tienen aplicaciones específicas según las necesidades del proyecto, ofreciendo ventajas únicas en distintos contextos de desarrollo.

Heredia López Betsy Aracely

Ambos tipos de lenguajes cumplen roles importantes en distintos contextos. Mientras que los lenguajes de scripting priorizan la agilidad y la facilidad de uso, los lenguajes funcionales destacan por su solidez teórica y capacidad de manejar la complejidad mediante principios como la inmutabilidad y las funciones puras.

Comprender las ventajas de cada enfoque permite a los desarrolladores elegir la herramienta adecuada para cada problema, combinar paradigmas de manera efectiva y diseñar soluciones más eficientes y sostenibles en el tiempo.

Hernández Silva Carlos Yahir

La programación funcional nos ayuda a escribir software más legible, conciso y fácil de probar. Al trabajar con funciones puras y evitar efectos secundarios, el código se vuelve más predecible y mantenible. Nos enfocamos en qué se quiere lograr, no en cómo hacerlo paso a paso, lo que facilita tanto el desarrollo como la depuración.

Referencias Bibliográficas

- Bird, R., & Wadler, P. (1988). Introduction to Functional Programming. Prentice Hall.*
- Hughes, J. (1989). Why functional programming matters. The Computer Journal, 32(2), 98-107. <https://doi.org/10.1093/comjnl/32.2.98>*
- Armstrong, J. (2007). Programming Erlang: Software for a Concurrent World. Pragmatic Bookshelf.*
- Odersky, M., Spoon, L., & Venners, B. (2010). Programming in Scala: A Comprehensive Step-by-Step Guide. Artima Press.*
- ¿Qué es la Programación Funcional? - Blog de Código Facilito. (n.d.). CódigoFacilito. <https://codigofacilito.com/articulos/programacion-funcional>*
- Corvo, H. S. (2020, March 19). Programación funcional: características, ejemplos, ventajas, desventajas. Lifeder. <https://www.lifeder.com/programacion-funcional/>*
- Sussman, G. J., & Steele, G. L. (1975). Scheme: An interpreter for extended lambda calculus. MIT AI Memo 349. <https://dspace.mit.edu/handle/1721.1/5794>*